

Ado. Net Entity Framework

Module

- Data Access in .NET
- Introduction to ADO.NET Entity Framework
- Implementing Database First Approach
- Implementing Model First Approach
- Implementing Code First Approach
- Summary

Data Access in .NET

- Ado.Net
- Linq
- EntityFramework

ADO. Net

- An evolutionary, more flexible successor to ADO
- A system designed for disconnected environments
- A programming model with advanced XML support
- A set of classes, interfaces, structures, and enumerations that manage data access from within the .NET Framework

ADO.NET

- Lack of intellisense support
- Results are untyped rectangular records
- Simple Interaction requires lengthy piece of code
- Managing Connections Becomes Tedious task

LinqToSql

- **LinqToSql** was Introduced to overcome drawback of ADO.NET
- Easy and faster way to interact with database
- ORM Mapping Tool
- Restricted to only SQL Server
- Does not support all types of relationship

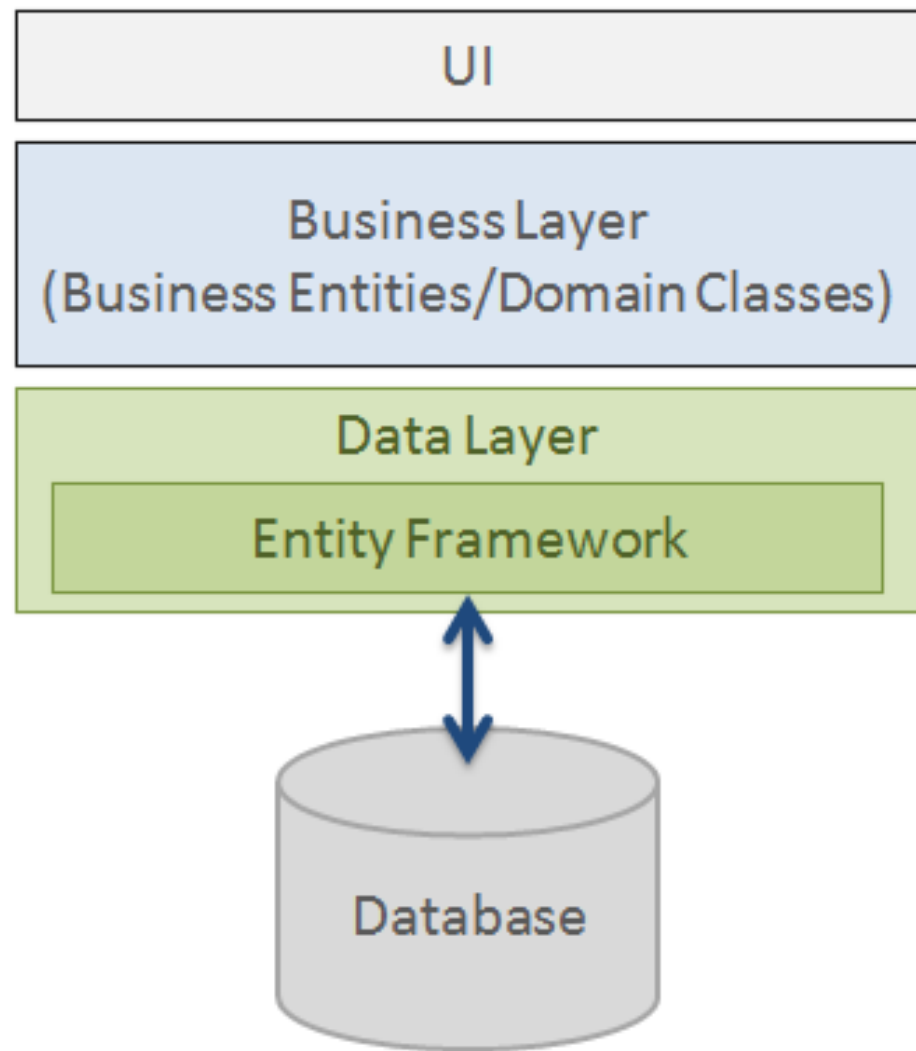
ADO.Net Entity Framework

EntityFramework = Linq to Sql + Extra Features

Multiple Data Sources

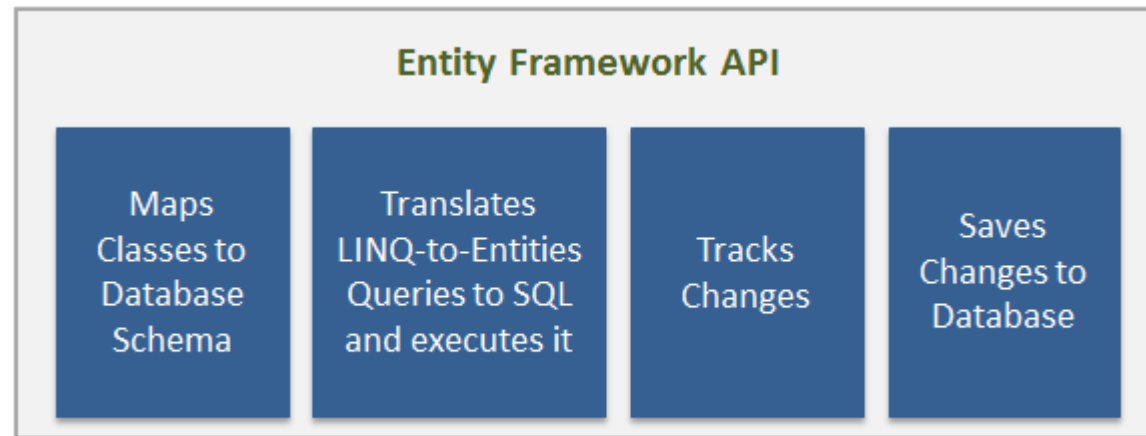
Ability to Generate Tables From Model

Support all kind of relationship



How Entity Framework Works

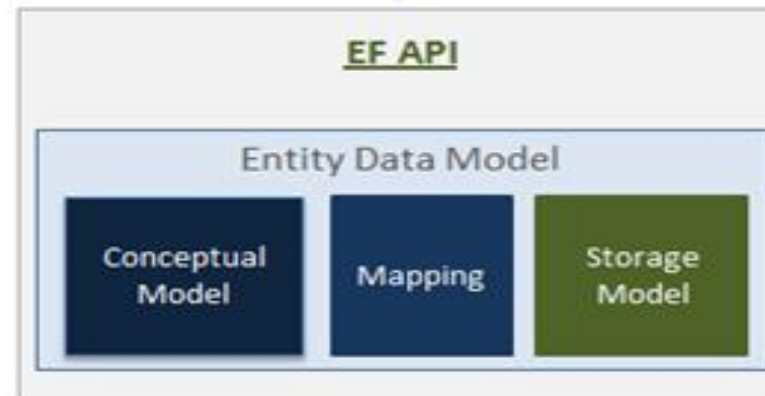
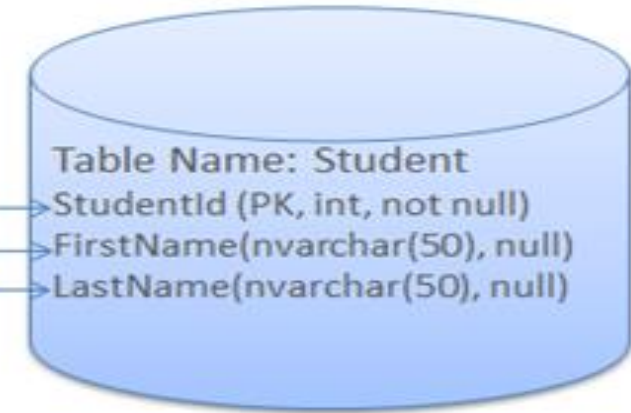
- Entity Framework API includes the ability to map domain (entity) classes to the database schema, translate & execute LINQ queries to SQL, track changes occurred on entities during their lifetime, and save changes to the database.



Entity Data Model:

```
public class Student
{
    public int StudentId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

© EntityFrameworkTutorial.net

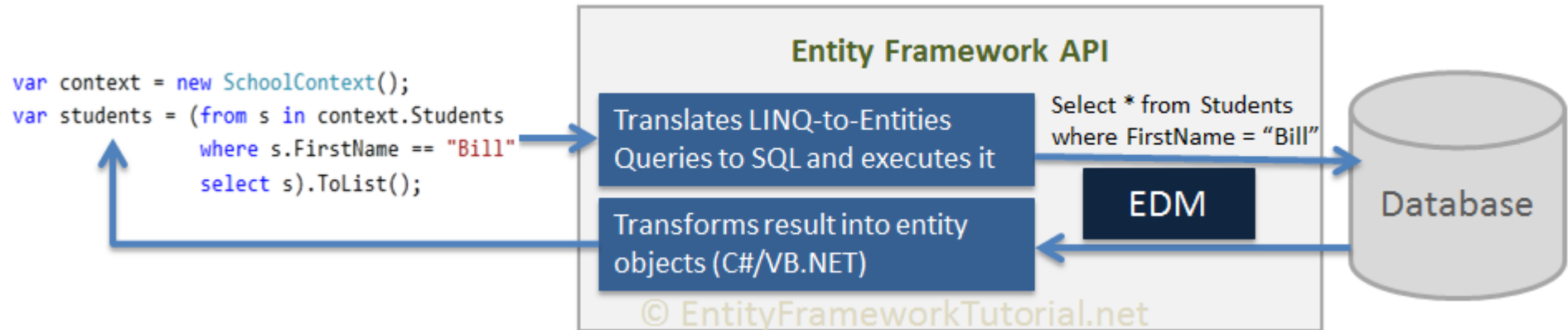


Entity Data Model:

- **Conceptual Model:** EF builds the conceptual model from your domain classes, context class, default conventions followed in your domain classes, and configurations.
- **Storage Model:** EF builds the storage model for the underlying database schema. In the code-first approach, this will be inferred from the conceptual model. In the database-first approach, this will be inferred from the targeted database.
- **Mappings:** EF includes mapping information on how the conceptual model maps to the database schema (storage model).

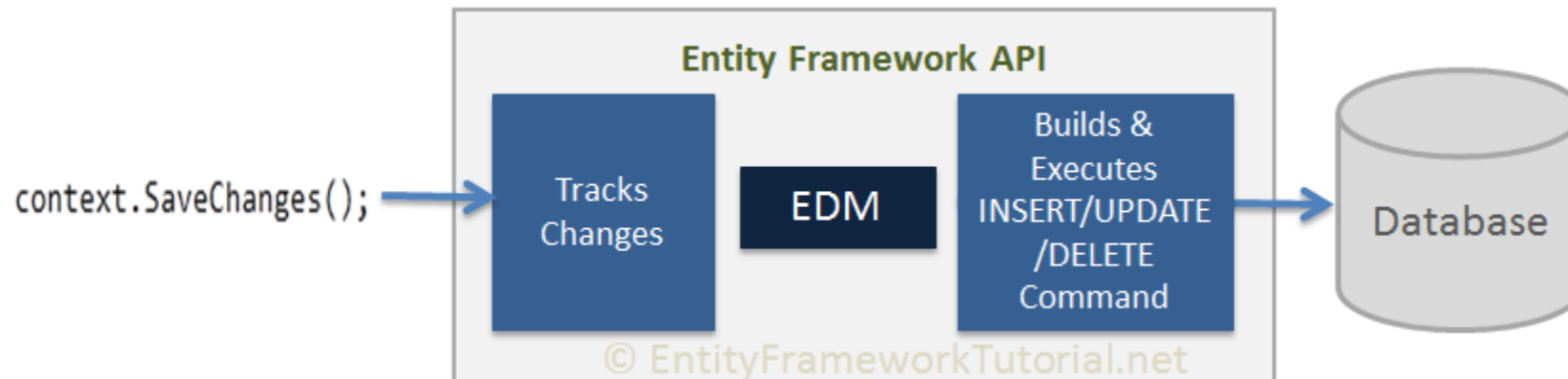
Querying

- EF API translates LINQ-to-Entities queries to SQL queries for relational databases using EDM and also converts results back to entity objects.



Saving

- EF API infers INSERT, UPDATE, and DELETE commands based on the state of entities when the `SaveChanges()` method is called. The `ChangeTrack` keeps track of the states of each entity as and when an action is performed.



- The context class in Entity Framework is a class which derives from DbContext in EF 6 and EF Core both. It is an important class in Entity Framework, which represents a session with the underlying database.
- The following SchoolContext class is an example of a context class.

Context Class in Entity Framework:

```
public class SchoolContext : DbContext
{
    public SchoolContext()
    {

    }

    public DbSet<Student> Students { get; set; }
    public DbSet<StudentAddress> StudentAddresses { get; set; }
    public DbSet<Grade> Grades { get; set; }
}
```

The context class is used to query or save data to the database. It is also used to configure domain classes, database related mappings, change tracking settings, caching, transaction etc.

What is an Entity in Entity Framework?

- An entity in Entity Framework is a class in the domain of your application which is included as a `DbSet<TEntity>` type property in the derived context class. EF API maps each entity to a table and each property of an entity to a column in the database.

```
public class Student
{
    public int StudentID { get; set; }
    public string StudentName { get; set; }
    public decimal Height { get; set; }
    public float Weight { get; set; }
    public Grade Grade { get; set; }
}
```


- The above classes become entities when they are included as `DbSet<TEntity>` properties in a context class (the class which derives from `DbContext`), as shown below.
- In context class, `Students` properties of type `DbSet<TEntity>` are called entity sets. The `Student`, `StudentAddress`, and `Grade` are entities (also known as entity types).

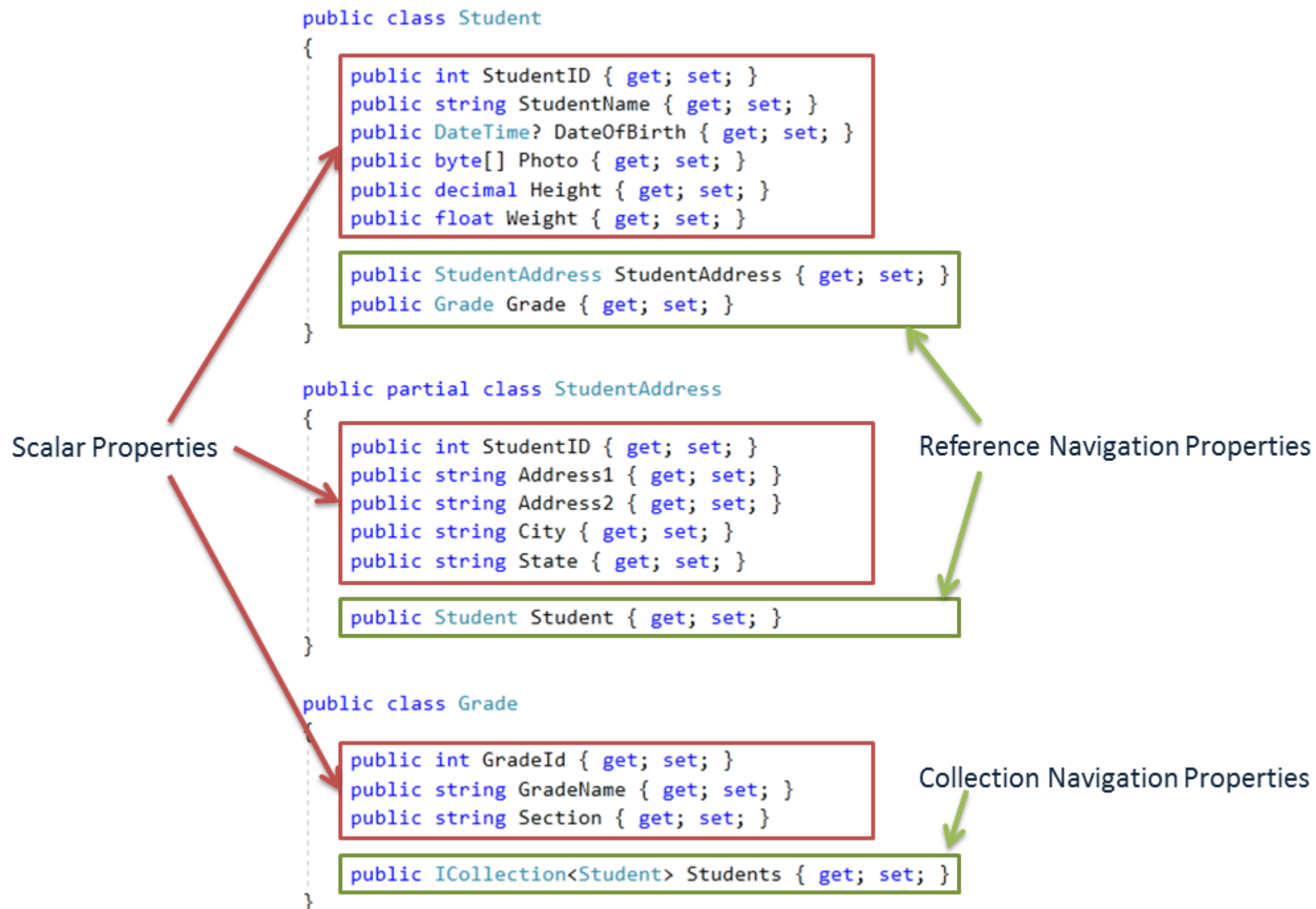
```
public DbSet<Student> Students { get; set; }
```

Scalar Properties and Navigation Properties.

- Scalar Property:
- The primitive type properties are called scalar properties. A scalar property stores the actual data. a scalar property maps to a single column in the database table.
- Navigation Property:
- The navigation property represents a relationship to another entity, as defined by an association. They do not carry data value.
- There are two types of navigation properties: Reference Navigation and Collection Navigation

navigation properties

- Reference Navigation Property:
 - If an entity includes a property of entity type, it is called a Reference Navigation Property. It represents multiplicity of one (1).
- Collection Navigation Property:
 - If an entity includes a property of collection type, it is called a collection navigation property. It represents multiplicity of many (*).



- The Context not only holds the reference to all the entity objects as soon as retrieved from the database, but also keeps track of entity states and maintains modifications made to the properties of the entity. This feature is known as *Change Tracking*.

Entity States:

- EF API maintains the state of each entity during an its lifetime. Each entity has a state based on the operation performed on it via the context class. The entity state represented by an enum `System.Data.Entity.EntityState`



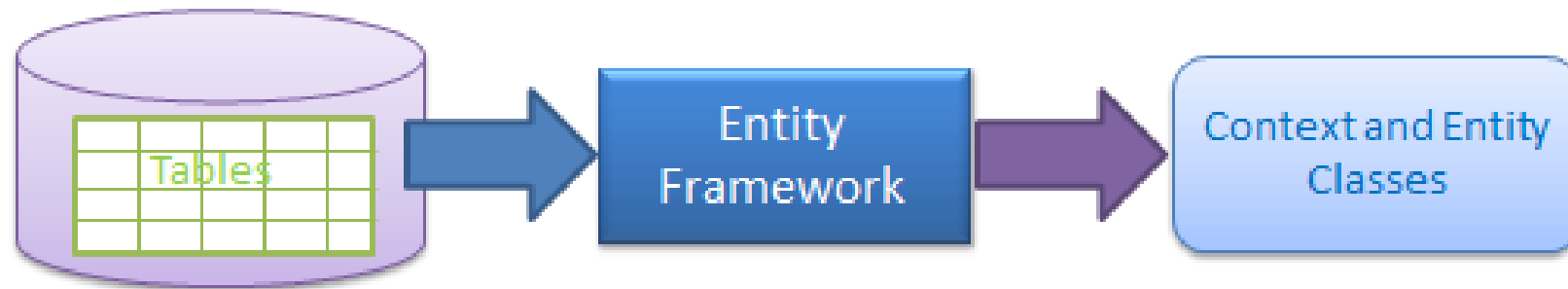
Added
Modified
Deleted
Unchanged

Development Approaches with Entity Framework:

- There are three different approaches you can use while developing your application using Entity Framework:
- Database-First
- Code-First
- Model-First

Database-First Approach:

- In the database-first development approach, you generate the context and entities for the existing database using EDM wizard integrated in Visual Studio or executing EF commands.

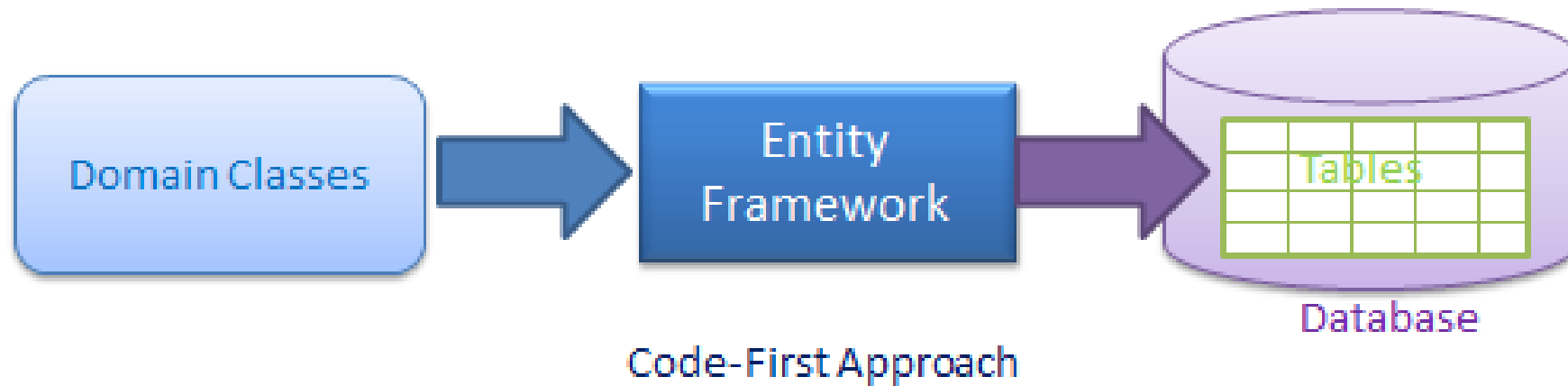


Database-First Approach

Code-First Approach:

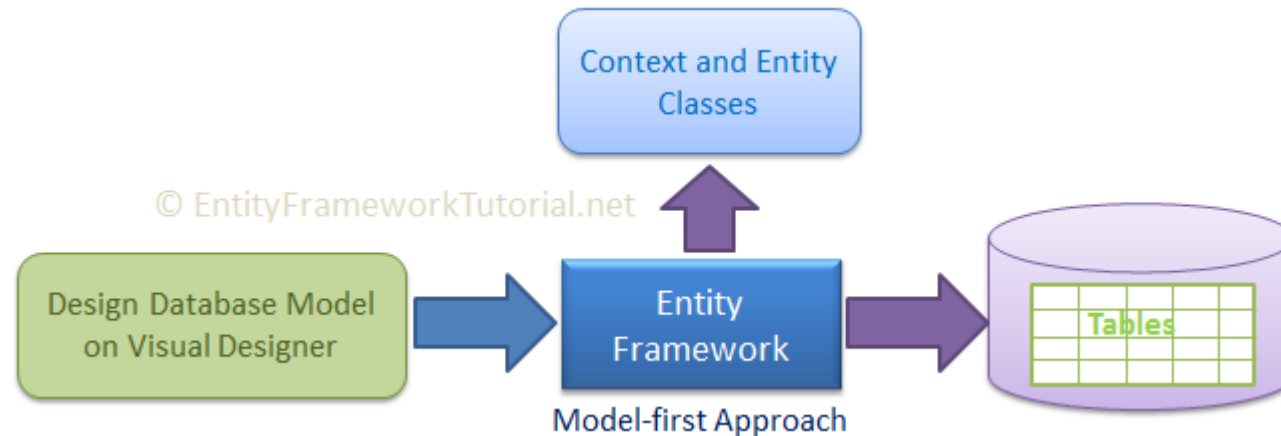
- Use this approach when you do not have an existing database for your application. In the code-first approach, you start writing your entities (domain classes) and context class first and then create the database from these classes using migration commands.
- Developers who follow the Domain-Driven Design (DDD) principles, prefer to begin with coding their domain classes first and then generate the database required to persist their data.

Code-First Approach:

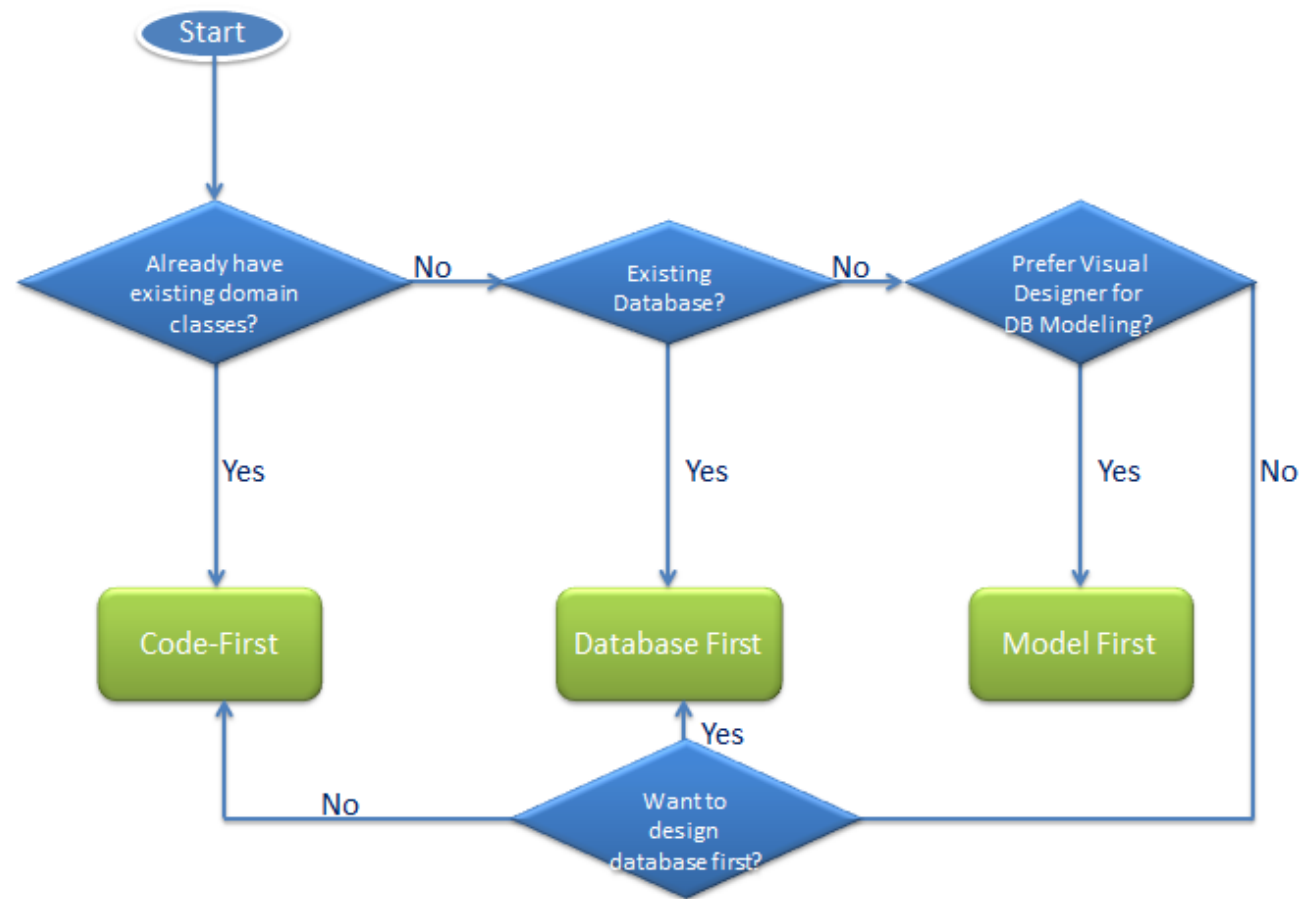


Model-First Approach:

- In the model-first approach, you create entities, relationships, and inheritance hierarchies directly on the visual designer integrated in Visual Studio and then generate entities, the context class, and the database script from your visual model.



Scenario



Data Annotations

- Data Annotations attributes are .NET attributes which can be applied on an entity class or properties to override default conventions in EF 6 and EF Core.
- Data annotation attributes are included in `System.ComponentModel.DataAnnotations`. These attributes are not only used in Entity Framework but they can also be used with ASP.NET MVC or data controls

Attribute	Description
Key	Can be applied to a property to specify a key property in an entity and make the corresponding column a PrimaryKey column in the database.
Timestamp	Can be applied to a property to specify the data type of a corresponding column in the database as rowversion.
ConcurrencyCheck	Can be applied to a property to specify that the corresponding column should be included in the optimistic concurrency check.
Required	Can be applied to a property to specify the corresponding column is a NotNull column in the database.
MinLength	Can be applied to a property to specify the minimum string length allowed in the corresponding column in the database.
MaxLength	Can be applied to a property to specify the maximum string length allowed in the corresponding column in the database.
StringLength	Can be applied to a property to specify maximum string length allowed corresponding column in the database.

